

---

# COLLABORATIVE FAULT TOLERANCE IN FOG COMPUTING USING REVERSE AUCTION AND GOSSIP

---

**Abdus Saboor Gaffari**

Department of Computer Science and Engineering  
College of Engineering  
American University of Sharjah  
Sharjah, UAE  
b00105302@aus.edu

**Solomon Ghebretatios**

Department of Computer Science and Engineering  
College of Engineering  
American University of Sharjah  
Sharjah, UAE  
b00104390@aus.edu

**Affan Murtaza**

Department of Computer Science and Engineering  
College of Engineering  
American University of Sharjah  
Sharjah, UAE  
b00104949@aus.edu

December 1, 2025

## ABSTRACT

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

## 1 Introduction

## 2 Related Work

## 3 Methodology / Materials and Methods

### 3.1 Proposed Approach

The proposed framework introduces a collaborative fault-tolerant model for fog computing in which autonomous fog nodes cooperate to maintain service continuity when failures are predicted or detected. The system operates within a multi-tier edge-fog-cloud hierarchy, where fog nodes act as the primary execution layer and collaborate only when necessary, without relying on any centralized controller or forced offloading. Collaboration emerges through decentralized, voluntary interactions between independently administered fog nodes, enabling both proactive FT through early migration and reactive FT when faults occur unexpectedly.

Two core mechanisms underpin the collaborative behavior. First, a gossip-based state dissemination protocol allows each fog node to gradually build an eventually complete and consistent view of the global resource state. By exchanging periodic summaries of CPU, memory, and bandwidth loads across a lightweight logical topology, fog nodes gain situational awareness without broadcasting costly global messages. This state is not required to be perfectly synchronized; instead, it serves as a sufficiently accurate substrate for decentralized decision-making during fault recovery. Second, a reverse-auction mechanism enables multi-owner fog nodes to negotiate container placements when local

recovery is not possible. A node experiencing a predicted or actual fault initiates an auction in which candidate fog nodes respond with bids reflecting their capacity and estimated migration cost. Collaborative FT emerges because nodes voluntarily participate, compete, and are compensated for accepting foreign containers, unlike forced or centrally dictated offloading.

The fault-tolerance workflow operates through a concise three-stage migration strategy: (i) intra-fog migration attempts to relocate containers within the same fog node whenever a healthy server is available; (ii) if local recovery is infeasible, inter-fog migration is triggered, where candidate fog nodes are evaluated using gossip-informed scoring and reverse-auction bidding; and (iii) if no suitable collaborator exists, the task is escalated to the cloud as a final fallback.

The framework executes tasks in lightweight, migratable containers to allow seamless pausing, checkpointing, transfer, and resumption across servers or fog nodes. This ensures that when a fault is predicted, all containers on the affected server are immediately checkpointed and migrated. If an unexpected fault occurs, the resulting behavior depends on the fault type: CPU and bandwidth degradations still permit checkpointing, whereas crashes cause complete loss of progress, requiring immediate restart elsewhere.

By combining container-based execution, gossip-driven distributed awareness, and reverse-auction-based negotiation, the proposed system provides a decentralized, prediction-aware, and economically aligned FT architecture suitable for heterogeneous, multi-owner fog environments.

### 3.2 System Model

The system consists of three interconnected layers: edge devices, fog nodes, and a cloud layer that provides a global but slower and more resource-intensive fallback. As shown in the architectural model, each fog node hosts multiple heterogeneous servers managed by a local control layer and an FT predictor module. The fog layer forms the main execution substrate, processing tasks originating from stationary edge devices. These devices continuously generate indivisible tasks that must run to completion and therefore cannot be split or distributed across servers. Each task executes inside a dedicated container, and each server can host multiple such containers concurrently, constrained only by its CPU, RAM, and bandwidth capacities. Containers can be paused, checkpointed, migrated, and resumed across any fog server or cloud resource, allowing mobility in response to failures.

The system assumes a multi-owner fog environment, where each fog node is controlled by an independent administrator. Nodes may strategically misreport internal metrics such as CPU load, memory usage, bandwidth availability, or expected migration time. However, they cannot fake observable outcomes, including actual migration duration, SLA adherence, or execution-time overload. This distinction is essential: misreporting affects negotiation, but actual behavior determines accountability. The framework embeds this assumption directly into its FT logic rather than treating it as an external constraint.

Tasks arriving at a fog node are placed onto the best available server based on that node’s own local scheduling policy. The proposed FT framework does not modify this placement algorithm; instead, it activates only when faults occur or are predicted. The FT predictor is assumed to exist and guarantees perfect predictions: no false positives or false negatives. Thus, every predicted fault will occur, and any server not flagged by the predictor is considered safe until a new prediction arises.

Three types of faults are considered: CPU degradation, bandwidth degradation, and server crash. While these faults differ in operational impact, the FT model treats them uniformly in terms of triggering migration, except for one distinction: crashes render servers completely inaccessible, causing absolute loss of container progress, whereas degradations allow checkpointing before migration. The fog-to-cloud communication path exhibits significantly higher latency and lower bandwidth than inter-fog communication links, reinforcing the preference for local or inter-fog recovery before escalating to the cloud.

This system model enables decentralized interaction among fog nodes, preserves each operator’s autonomy, supports truthful or untruthful behavior within defined limits, and leverages containerization to provide strong fault-migration guarantees across the hierarchy.

### 3.3 Gossip Protocol

The proposed collaborative FT system requires a lightweight and scalable mechanism for disseminating resource information among fog nodes, since no central coordinator is available to maintain global visibility. To meet this requirement, the framework adopts an eventual-consistency gossip process that enables each fog node to maintain an approximate yet sufficiently current understanding of cluster-wide resource availability without resorting to full

broadcast. This is essential because the architecture must support multi-owner environments where information exchange cannot impose excessive overhead or rely on full-system synchronization.

In this design, each fog node periodically snapshots its internal load conditions using the normalized CPU, memory, and bandwidth loads

$$L_{cpu}^i = \frac{\sum_{s \in S_i} U_{cpu}^s}{\sum_{s \in S_i} C_{cpu}^s} \quad (1)$$

$$L_{mem}^i = \frac{\sum_{s \in S_i} U_{mem}^s}{\sum_{s \in S_i} C_{mem}^s} \quad (2)$$

$$L_{bw}^i = \frac{\sum_{s \in S_i} U_{bw}^s}{\sum_{s \in S_i} C_{bw}^s} \quad (3)$$

and encapsulates them with a timestamp before forwarding the entry to its designated ring neighbor. The choice of a ring topology in our proposed system is deliberate: it limits each gossip cycle to a single outbound exchange, thereby reducing the probability of network storms while still ensuring that resource information propagates across the fog layer over successive rounds. Algorithm 1 presents the complete gossip dissemination process executed periodically at each fog node.

Since fog nodes belong to independent owners, the proposed system also allows the possibility of strategic misreporting within gossip exchanges. A node may intentionally publish understated load values in an attempt to attract incoming containers. However, the architecture is designed so that such misrepresentation becomes detectable indirectly through the trust scoring mechanism, where SLA performance and actual migration behavior reveal inconsistencies between claimed and observed behavior. The gossip process therefore functions as a decentralized substrate that the proposed collaborative FT system depends upon, not because it enforces honesty outright, but because it provides enough distributed visibility for reverse auction and trust scoring to operate effectively.

### 3.4 Reverse Auction for Collaborative Migration

To support voluntary cooperation between independently administered fog nodes, the proposed framework relies on a reverse-auction mechanism for inter-fog migration decisions. This is necessary because no fog node can be compelled to accept external work, and because multi-owner environments require a structured negotiation protocol rather than centralized placement. The reverse auction creates an economic and operational interface through which candidate nodes express their willingness to host an incoming container and articulate the cost associated with doing so.

When a predicted or detected fault prevents local recovery, the originating fog node issues a migration request to selected candidates determined via gossip-based scoring. Each candidate evaluates the request by estimating its migration cost, beginning with a claimed migration time

$$T_{mig}^{claimed} = t_{pause} + \frac{S_{container}}{Claimed_{bw}/8} + t_{resume} \quad (4)$$

where  $Claimed_{bw}$  represents the bandwidth the node asserts it can provide. The second component of the bid measures the resource impact of hosting the container according to the formulation

$$Impact = \frac{R_{cpu}}{C_{cpu}} + \frac{R_{mem}}{C_{mem}} + \frac{R_{bw}}{Claimed_{bw}} \quad (5)$$

leading to the bid

$$Bid_{value} = T_{mig}^{claimed} + k \cdot Impact \quad (6)$$

This structure is central to the proposed system because it integrates both migration overhead and local resource pressure into a unified cost, permitting fog nodes with limited capacity to discourage additional workload through naturally higher bids. Algorithm 2 formalizes the bid construction process at each candidate fog node.

Bid evaluation at the origin fog node incorporates the trust mechanism, which the proposed system uses to correct for the possibility of dishonest reporting. The effective bid

$$Bid^{eff}(i) = \frac{Bid_{value}(i)}{\max(\varepsilon, t(i))} \quad (7)$$

ensures that nodes with lower trust appear more expensive, thereby reducing their chances of being selected. This aligns incentives: fog nodes that consistently misreport resources or migration conditions become less competitive over time, while reliable nodes gradually dominate the market for hosting tasks. The reverse auction, combined with trust-adjusted bidding, thus provides the collaborative backbone required for decentralized FT, enabling safe, incentive-compatible inter-fog cooperation. The complete bid evaluation and winner selection procedure at the origin node is detailed in Algorithm 3.

**Algorithm 1** Gossip State Dissemination at Fog Node  $i$ 


---

```

1: Input: Set of servers  $S_i$ , state table  $StateTable_i$ , neighbor list  $N_i$ , current time  $t$ 
2: Output: Updated state tables at neighboring fog nodes
3:
4: // Compute local load snapshot
5:  $activeServers \leftarrow 0$ 
6:  $L_{cpu}^i \leftarrow 0, L_{mem}^i \leftarrow 0, L_{bw}^i \leftarrow 0$ 
7: for each server  $s \in S_i$  do
8:   if  $s$  is not faulted and not predicted to fail then
9:      $L_{cpu}^i \leftarrow L_{cpu}^i + \frac{U_{cpu}^s}{C_{cpu}^s}$ 
10:     $L_{mem}^i \leftarrow L_{mem}^i + \frac{U_{mem}^s}{C_{mem}^s}$ 
11:     $L_{bw}^i \leftarrow L_{bw}^i + \frac{U_{bw}^s}{C_{bw}^s}$ 
12:     $activeServers \leftarrow activeServers + 1$ 
13:   end if
14: end for
15: if  $activeServers > 0$  then
16:    $L_{cpu}^i \leftarrow L_{cpu}^i / activeServers$ 
17:    $L_{mem}^i \leftarrow L_{mem}^i / activeServers$ 
18:    $L_{bw}^i \leftarrow L_{bw}^i / activeServers$ 
19: else
20:    $L_{cpu}^i \leftarrow 1, L_{mem}^i \leftarrow 1, L_{bw}^i \leftarrow 1$ 
21: end if
22:
23: // Create local state entry with timestamp
24:  $entry_i \leftarrow \text{GossipStateEntry}(L_{cpu}^i, L_{mem}^i, L_{bw}^i, t)$ 
25:  $StateTable_i[i] \leftarrow entry_i$ 
26:
27: // Propagate state table to ring neighbors
28: for each neighbor  $j \in N_i$  do
29:   Send  $StateTable_i$  to fog node  $j$ 
30: end for
31:
32: // Upon receiving state table  $StateTable_k$  from fog node  $k$ :
33: for each entry  $(nodeId, state) \in StateTable_k$  do
34:   if  $nodeId \notin StateTable_i$  or  $state.timestamp > StateTable_i[nodeId].timestamp$  then
35:      $StateTable_i[nodeId] \leftarrow state$ 
36:   end if
37: end for

```

---

**3.5 Trust Scoring and Reputation Evolution**

Because the proposed collaborative FT mechanism assumes that fog nodes are independently owned and may behave strategically, a robust trust scoring process is required to maintain truthful behavior across gossip exchanges and bidding interactions. Trust values  $t(i) \in [0, 1]$  quantify each node's historical reliability and are updated after every completed task using both migration-stage and execution-stage measurements. This is essential for enabling collaboration without imposing central oversight. Every fog node maintains a trust value for every other fog node in the network.

The trust mechanism evaluates explicit bid honesty by comparing claimed versus actual migration performance. The actual bandwidth is reconstructed from observed transfer behavior as

$$Actual_{bw} = \frac{S_{container}}{T_{transfer}} \times 8 \quad (8)$$

and the relative bandwidth honesty error is

$$e_{bw} = \frac{|Claimed_{bw} - Actual_{bw}|}{\max(\varepsilon, Claimed_{bw})} \quad (9)$$

**Algorithm 2** Bid Calculation at Candidate Fog Node  $i$ 


---

```

1: Input: Migration request for container  $c$  from origin fog node  $j$ 
2: Output: Bid response containing feasibility and bid value
3:
4: // Retrieve container requirements
5:  $R_{cpu} \leftarrow c.demandMips$ 
6:  $R_{mem} \leftarrow c.ramMb$ 
7:  $R_{bw} \leftarrow c.bandwidth$ 
8:  $S_{container} \leftarrow c.containerSizeMb$ 
9:  $D_{sla} \leftarrow c.deadlineSeconds$ 
10:
11: // Compute claimed migration time
12:  $Claimed_{bw} \leftarrow$  available bandwidth to fog node  $j$  (may be strategically reported)
13:  $T_{transfer} \leftarrow S_{container}/(Claimed_{bw}/8)$ 
14:  $T_{mig}^{claimed} \leftarrow t_{pause} + T_{transfer} + t_{resume}$ 
15:
16: // Compute projected resource load
17:  $L_{cpu} \leftarrow$  current CPU load at fog node  $i$ 
18:  $L_{mem} \leftarrow$  current memory load at fog node  $i$ 
19:  $L_{bw} \leftarrow$  current bandwidth load at fog node  $i$ 
20:  $projected_{cpu} \leftarrow L_{cpu} + R_{cpu}/C_{cpu}$ 
21:  $projected_{mem} \leftarrow L_{mem} + R_{mem}/C_{mem}$ 
22:  $projected_{bw} \leftarrow L_{bw} + R_{bw}/Claimed_{bw}$ 
23:
24: // Check feasibility
25:  $timeToDeadline \leftarrow \max(0, D_{sla} - t)$ 
26:  $hasHealthyServer \leftarrow$  any server at  $i$  is not faulted and not predicted to fail
27:  $feasible \leftarrow hasHealthyServer$  and  $projected_{cpu} \leq 1$  and  $projected_{mem} \leq 1$ 
28:    and  $projected_{bw} \leq 1$  and  $T_{mig}^{claimed} \leq timeToDeadline$ 
29:
30: // Compute bid value
31:  $Impact \leftarrow \frac{R_{cpu}}{C_{cpu}} + \frac{R_{mem}}{C_{mem}} + \frac{R_{bw}}{Claimed_{bw}}$ 
32:  $Bid_{value} \leftarrow T_{mig}^{claimed} + k \cdot Impact$   $\triangleright k$  is resource impact weight
33:
34: // Return bid response
35: return BidResponse( $i, c.id, feasible, Bid_{value}, Claimed_{bw}, T_{mig}^{claimed}$ )

```

---

Similarly, the discrepancy between claimed and actual migration times is captured through

$$T_{mig}^{actual} = t_{pause} + T_{transfer} + t_{resume} \quad (10)$$

$$e_{mig} = \frac{|T_{mig}^{claimed} - T_{mig}^{actual}|}{\max(\varepsilon, T_{mig}^{claimed})} \quad (11)$$

In the trust model, the parameters  $\tau_{bw}$  and  $\tau_{mig}$  define the dishonesty thresholds for bandwidth and migration time, respectively. Error values above these thresholds indicate intentional misreporting. The trust update therefore applies a multiplicative decay using

$$t'(i) = t(i) \cdot \delta_{decay} \quad (12)$$

whenever  $e_{bw} > \tau_{bw}$  or  $e_{mig} > \tau_{mig}$ .

To detect dishonest execution behavior such as understated CPU or memory load, the framework evaluates SLA performance using

$$m_{sla} = \frac{D_{sla} - T_{complete}}{\max(\varepsilon, D_{sla})} \quad (13)$$

Negative margins indicate SLA violations. The additive penalty

$$t''(i) = \max(0, t'(i) - \beta_{violation}) \quad (14)$$

**Algorithm 3** Bid Evaluation and Winner Selection at Origin Fog Node

---

```

1: Input: Container  $c$ , set of bid responses  $Bids$ , trust scores  $T$ , trust threshold  $\theta_{trust}$ 
2: Output: Selected winner fog node or cloud fallback
3:
4:  $FeasibleBids \leftarrow \emptyset$ 
5: for each bid  $b \in Bids$  do
6:    $bidderId \leftarrow b.bidderId$ 
7:    $t(bidderId) \leftarrow T[bidderId]$ 
8:
9:   // Filter by trust threshold
10:  if trust enabled and  $t(bidderId) < \theta_{trust}$  then
11:    continue ▷ Exclude low-trust nodes
12:  end if
13:
14:  // Verify feasibility using local knowledge
15:  if not  $b.feasible$  then
16:    continue
17:  end if
18:
19:  // Compute effective bid using trust adjustment
20:   $Bid^{eff}(bidderId) \leftarrow \frac{b.bidValue}{\max(\epsilon, t(bidderId))}$ 
21:
22:  // Compute projected headroom for tie-breaking
23:   $headroom(bidderId) \leftarrow (1 - projected_{cpu}) + (1 - projected_{mem}) + (1 - projected_{bw})$ 
24:
25:   $FeasibleBids \leftarrow FeasibleBids \cup \{(bidderId, Bid^{eff}, headroom, latency)\}$ 
26: end for
27:
28: // Select winner using lexicographic ordering
29: if  $FeasibleBids \neq \emptyset$  then
30:   Sort  $FeasibleBids$  by ( $Bid^{eff}$  ascending,  $headroom$  descending,  $latency$  ascending)
31:    $winner \leftarrow$  first element in sorted  $FeasibleBids$ 
32:   return  $winner.bidderId$ 
33: else
34:   return cloud fallback
35: end if

```

---

is applied when  $m_{sla} < -\tau_{sla}$ , where  $\tau_{sla}$  defines the tolerance for SLA deviation. The parameter  $\beta_{violation}$  quantifies how strongly such violations reduce trust.

Conversely, the trust model rewards consistently honest fog nodes through

$$t''(i) = \min(1, t'(i) + \beta_{success}) \quad (15)$$

whenever all honesty conditions are satisfied. The parameter  $\beta_{success}$  governs the incremental trust gained from compliant behavior.

The final trust value is

$$t(i) = t''(i) \quad (16)$$

which integrates both migration-stage honesty checks and SLA-based execution verification. The parameters  $\tau_{bw}$ ,  $\tau_{mig}$ , and  $\tau_{sla}$  therefore define the boundaries of acceptable error, while  $\beta_{violation}$  and  $\beta_{success}$  control how penalties and rewards shape long-term trust evolution. Together, these mechanisms ensure that repeated dishonesty, whether through gossip, bidding, or execution, reduces a node's influence in future auctions, while sustained honest behavior is economically reinforced.

In addition to these active updates, trust also improves passively over time so that nodes that have not recently hosted tasks can gradually regain competitiveness. The recovery magnitude is proportional to the remaining distance to perfect trust, and at periodic intervals the system applies

$$t_{passive}(i, t) = \min(1, t(i) + r_{passive} \cdot (1 - t(i))) \quad (17)$$

**Algorithm 4** Trust Evaluation and Update with Passive Recovery

---

```

1: Input: Fog node  $i$ , container  $c$ , actual metrics ( $Actual_{bw}, T_{mig}^{actual}$ ),
2:   claimed metrics ( $Claimed_{bw}, T_{mig}^{claimed}$ ), completion time  $T_{complete}$ 
3: Output: Updated trust score  $t(i)$ 
4:
5: // Phase 1: Migration and bandwidth honesty evaluation
6:  $e_{bw} \leftarrow \frac{|Claimed_{bw} - Actual_{bw}|}{\max(\varepsilon, Claimed_{bw})}$ 
7:  $e_{mig} \leftarrow \frac{|T_{mig}^{claimed} - T_{mig}^{actual}|}{\max(\varepsilon, T_{mig}^{claimed})}$ 
8:
9:  $t'(i) \leftarrow t(i)$  ▷ Initialize with current trust
10: if  $e_{bw} > \tau_{bw}$  or  $e_{mig} > \tau_{mig}$  then
11:    $t'(i) \leftarrow t(i) \cdot \delta_{decay}$  ▷ Apply multiplicative decay
12: end if
13:
14: // Phase 2: SLA-based evaluation
15:  $m_{sla} \leftarrow \frac{D_{sla} - T_{complete}}{\max(\varepsilon, D_{sla})}$ 
16:
17: if  $m_{sla} < -\tau_{sla}$  then
18:    $t''(i) \leftarrow \max(0, t'(i) - \beta_{violation})$  ▷ Apply additive penalty
19: else
20:   if  $e_{bw} \leq \tau_{bw}$  and  $e_{mig} \leq \tau_{mig}$  then
21:      $t''(i) \leftarrow \min(1, t'(i) + \beta_{success})$  ▷ Apply additive reward
22:   else
23:      $t''(i) \leftarrow t'(i)$  ▷ No SLA reward if migration dishonest
24:   end if
25: end if
26:
27:  $t(i) \leftarrow t''(i)$  ▷ Store updated trust
28:
29: // Passive recovery (applied periodically for all fog nodes)
30: procedure PASSIVERECOVERY( $i, t_{current}, t_{last\_recovery}$ )
31:   if  $t_{current} - t_{last\_recovery} \geq \Delta t_{recovery}$  then
32:      $increment \leftarrow r_{passive} \cdot (1 - t(i))$ 
33:      $t(i) \leftarrow \min(1, t(i) + increment)$ 
34:     Update  $t_{last\_recovery}(i) \leftarrow t_{current}$ 
35:   end if
36: end procedure
37:
38: return  $t(i)$ 

```

---

where  $r_{passive}$  is a small recovery rate applied at regular time intervals. This slow and asymptotic rise prevents long-term exclusion of inactive nodes while ensuring that previously dishonest nodes do not regain trust abruptly. The complete trust update mechanism, integrating both active evaluation and passive recovery, is formalized in Algorithm 4.

### 3.6 Migration Workflow

The migration workflow describes how the system reacts once a fault is predicted or detected and how it determines the next execution location for each affected container. Faults differ in severity, with some allowing clean checkpointing and others destroying progress, so the workflow adapts to the specific situation. It first attempts to keep the task within the same fog node, then expands the search to collaborating fog nodes, and finally relies on the cloud when no suitable collaborator is available. This process unifies feasibility checks, gossip visibility, dynamic scoring, reverse auction bidding, and trust adjustment into a single decision path. Algorithm 6 presents the complete migration decision process.

When the FT predictor flags a server as approaching failure, all containers on that server are paused and checkpointed. Unexpected CPU or bandwidth degradation still permits an orderly pause, while crashes erase all progress. Once

removed, each container enters the migration engine, which begins with local recovery. The local scheduler is queried for a healthy server. If one exists, the container resumes locally. If not, the workflow moves to inter-fog migration.

Here, the system must compare independently operated fog nodes using the state information disseminated through gossip. Dynamic scoring interprets this information in relation to the container's needs. The resource profile is expressed through proportional contributions

$$p_{cpu}, \quad p_{mem}, \quad p_{bw}, \quad (18)$$

which sum to one and describe whether the workload is primarily CPU, memory, or bandwidth oriented. Workload alone is insufficient, so urgency is added through the normalized slack factor

$$U_{norm} = 1 - \frac{\max(0, D_{sla} - T_{pred})}{D_{sla}}, \quad (19)$$

which increases as the task approaches its deadline.

Using both workload and urgency, the system computes the weights  $\alpha$ ,  $\beta$ , and  $\gamma$ . Since urgent tasks are more sensitive to bandwidth, the bandwidth weight becomes

$$\gamma = p_{bw} + U_{norm}(1 - p_{bw}), \quad (20)$$

and the remainder is divided between CPU and memory:

$$\alpha = (1 - \gamma) \frac{p_{cpu}}{p_{cpu} + p_{mem}}, \quad \beta = (1 - \gamma) \frac{p_{mem}}{p_{cpu} + p_{mem}}. \quad (21)$$

These weights, which sum to one, determine how heavily each load dimension influences the dynamic score. Fog nodes with stale gossip entries are discarded, and the highest scoring nodes become candidates for the next stage.

To combine these weights with the gossip-reported loads  $L_{cpu}^i$ ,  $L_{mem}^i$ , and  $L_{bw}^i$  of node  $i$ , the system computes the dynamic score

$$Score(i) = \alpha (1 - L_{cpu}^i) + \beta (1 - L_{mem}^i) + \gamma (1 - L_{bw}^i), \quad (22)$$

which reflects how well node  $i$  aligns with the container's CPU, memory, and bandwidth demands under its current urgency. Higher scores indicate more suitable collaborators for migration. The complete scoring procedure is formalized in Algorithm 5.

The reverse auction is then initiated. Each candidate constructs a bid using the previously defined equations for claimed migration time and resource impact. The origin node adjusts these bids using the trust based effective bid formula, ensuring that unreliable nodes naturally appear less attractive. The winning fog node is selected using the established lexicographic rule. Migration proceeds to that node and the system records the observed transfer duration as  $T_{transfer}$ . The receiving node's scheduler handles final server-level placement.

If no fog node is capable of hosting the container, the system escalates to the cloud, which does not participate in bidding. The transfer is completed,  $T_{transfer}$  is recorded, and the task resumes or restarts depending on whether checkpointing was possible.

Overall, the workflow creates a structured and flexible response to failures. It prioritizes local recovery, leverages distributed scoring and auction-based cooperation when needed, and uses the cloud only as a final fallback. This layered approach maintains task continuity across both predicted and unexpected fault scenarios, even in environments where fog nodes might misreport their state or behave strategically.

### 3.6.1 Payment Settlement

After a migrated task completes, the originating fog node settles the payment using the final payment rule. The payment is reduced when the hosting fog node is found to have misreported bandwidth or migration time, and it is also reduced when the task violates its SLA. The updated settlement is governed by

$$\text{Payment} = \max(0, Bid_{value} - P_{lie} - P_{sla}), \quad (23)$$

where dishonesty penalties are computed as

$$P_{lie} = \lambda(e_{bw} + e_{mig}), \quad (24)$$

and SLA penalties derive from the SLA margin defined earlier,

$$P_{sla} = \mu \cdot \max(0, -m_{sla}). \quad (25)$$

Here,  $\lambda$  controls the strength of dishonesty penalties and  $\mu$  controls the magnitude of SLA penalties. Intra-fog placements yield zero payment, and cloud placements follow the same rule while not contributing to trust adjustments. This settlement step ensures that economic rewards reflect honest reporting and timely execution, while also attaching a financial consequence to misleading bids and missed SLAs. The complete settlement process is detailed in Algorithm 7.



**Algorithm 5** Dynamic Scoring of Fog Nodes for Container Migration

---

```

1: Input: Container  $c$ , state table  $StateTable$ , staleness threshold  $\theta_{stale}$ , current time  $t$ 
2: Output: Ranked list of candidate fog nodes
3:
4: // Compute workload proportions
5:  $total \leftarrow R_{cpu} + R_{mem} + R_{bw}$ 
6:  $p_{cpu} \leftarrow R_{cpu}/total$ 
7:  $p_{mem} \leftarrow R_{mem}/total$ 
8:  $p_{bw} \leftarrow R_{bw}/total$ 
9:
10: // Compute normalized urgency
11:  $slack \leftarrow \max(\varepsilon, D_{sla} - t)$ 
12:  $U \leftarrow 1/slack$ 
13:  $U_{norm} \leftarrow \min(1.0, U \cdot k)$   $\triangleright k = 0.1$  is urgency scaling factor
14:
15: // Compute dynamic weights
16:  $\gamma \leftarrow p_{bw} + U_{norm}(1 - p_{bw})$ 
17:  $W_{remaining} \leftarrow 1 - \gamma$ 
18:  $\alpha \leftarrow W_{remaining} \cdot \frac{p_{cpu}}{p_{cpu} + p_{mem}}$ 
19:  $\beta \leftarrow W_{remaining} \cdot \frac{p_{mem}}{p_{cpu} + p_{mem}}$ 
20:
21: // Score all non-stale fog nodes
22:  $Candidates \leftarrow \emptyset$ 
23: for each fog node  $i$  in  $StateTable$  do
24:   if  $i$  is local node then
25:     continue
26:   end if
27:    $entry \leftarrow StateTable[i]$ 
28:   if  $t - entry.timestamp > \theta_{stale}$  then  $\triangleright$  Discard stale entries
29:     continue
30:   end if
31:    $Score(i) \leftarrow \alpha(1 - L_{cpu}^i) + \beta(1 - L_{mem}^i) + \gamma(1 - L_{bw}^i)$ 
32:    $Candidates \leftarrow Candidates \cup \{(i, Score(i))\}$ 
33: end for
34:
35: // Sort by score descending
36: Sort  $Candidates$  by score in descending order
37: return  $Candidates$ 

```

---

**3.7 Effectiveness of Solution**

This section explains how the proposed system behaves when fog nodes try to behave strategically or dishonestly. The explanations are written as different scenarios that walk through what happens in each case. The reasoning still comes from the mechanisms already described earlier, such as the bidding formulation, the gossip scoring, the trust update equations with the two-phase adjustments, and the SLA-based evaluation that happens naturally after execution. The goal here is not to introduce anything new, but just to show how the pieces already introduced end up supporting the system even when nodes do not behave nicely.

**Scenario 1: A fog node understates its bid.** If a fog node tries to understate its bid by claiming a smaller migration time or a lower resource impact than what it actually has, then it might look attractive at first. It may even win a bid occasionally because the origin node simply sees a lower cost. But as soon as the container finishes migration, the actual bandwidth and the actual migration duration are measured using the equations already given for  $B_{actual}$  and the real  $T_{mig,actual}$ . When the claimed numbers and the actual numbers do not match, the relative errors appear, and these errors immediately activate the multiplicative trust decay in Phase 1 of the trust update. And once that happens, the trust value decreases and the node's future effective bids become much higher because effective bids divide by trust. So the node that understates its bid ends up becoming less competitive later. This means the short-term gain is lost quickly, and in many cases it becomes much worse off. So understating a bid does not really help the node, and the system naturally pushes it out.

**Scenario 2: A fog node overstates its bid.** If a fog node overstates its bid and reports a much higher cost than necessary, the system does not punish it with trust decay because the actual measurements still match what was claimed. But this behavior still does not work out well for the node. The inflated bid simply places it at the bottom of the winner selection, and because winner selection always tries to reduce the effective bid, such a node almost never gets chosen. In some sense, the node excludes itself by being too expensive. This means overstating does not give any advantage, and it slowly pushes the node away from being part of the collaboration. The system does not need to punish it, because the bidding logic already does that indirectly.

**Scenario 3: A fog node lies about bandwidth or migration time.** A fog node may try to lie by putting nicer claimed bandwidth or shorter claimed migration times into its bid. But once the migration actually takes place, the real transfer bandwidth and the real migration duration can be computed directly from the observed transfer using the earlier equations. These measured values go into the relative error formulas, and when the discrepancy becomes noticeable, the thresholds for dishonesty are crossed. As soon as that happens the trust update does the multiplicative penalty, and sometimes the additive one too if the SLA is also harmed. And because trust affects the effective bid so strongly, even a few dishonest bids make the node almost impossible to select in future migrations. So lying about bandwidth or migration time is always detected eventually because we do not rely on the claimed values, we rely on what actually happened.

**Scenario 4: A fog node lies about CPU or RAM load.** CPU and memory dishonesty is a bit harder because these values cannot be checked directly during the bidding process. But they show up later through the SLA margin. If a fog node lies and says it has plenty of available CPU or RAM and then tries to run the container with less than promised, the execution slows down. The actual completion time becomes larger, and the SLA margin becomes negative, and this automatically triggers the additive trust penalty in Phase 2. If this continues to happen multiple times, the trust value keeps dropping. Once trust goes below the threshold, the node stops being considered at all. So even though the system does not directly check CPU or RAM claims, it relies on the SLA margin to expose dishonesty. And since SLA violations are observed facts, a fog node cannot hide the results of its lies.

**Scenario 5: A fog node lies in the gossip state.** A fog node may try to cheat by reporting a lighter load in the gossip state, hoping to look more attractive when scores are calculated. But gossip load values come from actual server measurements, so this type of lying is already constrained. And even if the node could somehow lower its reported load, it still has to execute the containers that it attracts. If it is actually overloaded, execution slows down and that leads to SLA violations. And these SLA violations directly activate the additive penalty in the trust update. So even if gossip lying helped in the short term, the resulting poor performance would appear very quickly, and trust would decrease. And since lower trust makes bidding worse, the node eventually stops being selected anyway. So lying in gossip does not really help, and eventually it becomes counterproductive.

**Scenario 6: A fog node uses multiple dishonest strategies together.** A fog node may think that if it mixes different types of lies, like fake gossip values, wrong CPU load reports, underestimated migration times, or overstated bandwidth, then maybe the system cannot catch all of it. But because the trust update works in two separate phases, and because SLA outcomes are always observable, the penalties accumulate. Migration dishonesty activates the multiplicative decay in Phase 1. SLA dishonesty activates the additive penalty in Phase 2. When both happen together, the trust falls even faster. Once trust becomes small, the effective bid becomes extremely large, and the node almost never gets selected. So combining dishonest behaviors does not hide anything; instead it accelerates exclusion from the collaboration.

These scenarios show, in a practical and scenario-driven way, that the mechanisms introduced earlier work together so that lying or misreporting is not a sustainable strategy. The system keeps moving toward nodes that behave honestly, because honest nodes naturally get better trust, better effective bids, and fewer penalties. The result is that the proposed FT system remains stable and cooperative even when nodes try to game the process.

### 3.8 Experimental Setup

The evaluation of the proposed collaborative FT approach was conducted using a customized version of iFogSim<sup>1</sup> that we modified to support all components of our design, including container-level execution, checkpointing, migration, gossip-driven state dissemination, urgency-aware scoring, reverse-auction bidding, and trust-based negotiation. These extensions allowed the simulator to faithfully reproduce the behavior of the proposed mechanisms rather than relying on the default iFogSim model. All workloads, failures, and communication events were generated synthetically within the modified simulator, enabling controlled experiments and full reproducibility without reliance on external datasets.

<sup>1</sup><https://github.com/Cloudslab/iFogSim>

Table 1 summarizes the system-level configuration used in all experiments. The simulated environment consists of ten fog nodes arranged in a logical ring for gossip communication. Each fog node hosts three to four heterogeneous servers with capacities in the range of  $C_{cpu}^s \in [2000, 2200]$  MIPS,  $C_{mem}^s = 4096$  MB, and  $C_{bw}^s \in [3500, 4500]$  Mbps. Each fog node is connected to six stationary edge devices that continuously generate tasks. Cloud resources were provisioned with significantly higher capacity to serve as a reliable fallback destination. Network delays follow the fog-computing hierarchy, where edge–fog links are fastest, inter-fog communication is moderately fast, and fog–cloud links are slower and more bandwidth-constrained.

Faults, workload parameters, bidding configuration, trust parameters, and SLA specification are summarized in Table 2. Faults are injected according to a Poisson arrival process with a mean inter-failure time of 60 seconds, and each fault includes a prediction window of 300 seconds and a recovery time of 240 seconds. The three supported fault types—CPU degradation, bandwidth degradation, and server crash—follow the probabilities listed in the table. CPU and bandwidth degradations allow the system to checkpoint containers before migrating them, while crashes cause complete loss of progress. During inter-fog migration, fog nodes respond with bids computed using the claimed migration time and resource impact formulations defined earlier, parameterized by the constants  $k$ ,  $t_{pause}$ , and  $t_{resume}$ . Trust scores evolve according to measured discrepancies in bandwidth and migration honesty, SLA behavior, and passive recovery, governed by the thresholds  $\tau_{bw}$ ,  $\tau_{mig}$ , and  $\tau_{sla}$ .

Task workloads were selected to produce realistic load conditions across the fog layer. Each edge device generates twenty tasks with exponential inter-arrival times, using configurable jitter. Resource requirements for tasks—including CPU, memory, bandwidth, runtime, and container size—are sampled from the value ranges shown in Table 2. SLA deadlines scale with task runtime according to the multiplier provided in the configuration.

A baseline system was implemented for comparison, identical to the proposed mode except that gossip dissemination, trust scoring, and auction-based collaboration were disabled. Instead, all migration decisions were handled by a centralized scheduler. This allowed direct comparison between our decentralized collaborative FT approach and a simplified centralized system under identical workload and fault conditions.

Table 1: System and Network Configuration

| Parameter                               | Value          |
|-----------------------------------------|----------------|
| Number of Fog Nodes                     | 10             |
| Servers per Fog Node                    | 3–4            |
| CPU Capacity per Server ( $C_{cpu}^s$ ) | 2000–2200 MIPS |
| Memory per Server ( $C_{mem}^s$ )       | 4096 MB        |
| Bandwidth per Server ( $C_{bw}^s$ )     | 3500–4500 Mbps |
| Storage per Server                      | 100000 MB      |
| Uplink/Downlink Bandwidth               | 6000–9000 Mbps |
| Cloud CPU Capacity                      | 9000 MIPS      |
| Cloud Memory                            | 45000 MB       |
| Cloud Bandwidth                         | 40000 Mbps     |
| Cloud Storage                           | 200000 MB      |
| Cloud Uplink/Downlink                   | 40000 Mbps     |
| Edge Devices per Fog Node               | 6              |
| Edge Latency                            | 3 ms           |
| Edge Bandwidth                          | 1500 Mbps      |
| Edge Heterogeneity Jitter               | 10%            |
| Inter-Fog Latency                       | 8 ms           |
| Inter-Fog Bandwidth                     | 1500 Mbps      |
| Fog-Cloud Latency                       | 70 ms          |
| Fog-Cloud Bandwidth                     | 150 Mbps       |
| Gossip Enabled                          | True           |
| Gossip Interval                         | 7 s            |
| Gossip Staleness Threshold              | 3 intervals    |

Table 2: Fault, Bidding, Trust, and Task Configuration

| Category                                        | Parameter and Value |
|-------------------------------------------------|---------------------|
| <b>Fault Model</b>                              |                     |
| Prediction Lead Time                            | 300 s               |
| Mean Time Between Failures                      | 60 s                |
| Fault Start Delay                               | 30 s                |
| Recovery Time                                   | 240 s               |
| CPU Failure Probability                         | 0.33                |
| Bandwidth Degradation Probability               | 0.34                |
| Server Crash Probability                        | 0.33                |
| <b>Bidding Model</b>                            |                     |
| Max Fog Bidders                                 | 4                   |
| Pause Time ( $t_{pause}$ )                      | 1.5 s               |
| Resume Time ( $t_{resume}$ )                    | 1.5 s               |
| Resource Impact Weight ( $k$ )                  | 0.6                 |
| <b>Trust Model</b>                              |                     |
| Trust Enabled                                   | True                |
| Decay Factor ( $\delta_{decay}$ )               | 0.5                 |
| Trust Recovery Factor                           | 0.05                |
| Trust Threshold                                 | 0.5                 |
| Bandwidth Dishonesty Threshold ( $\tau_{bw}$ )  | 0.3                 |
| Migration Dishonesty Threshold ( $\tau_{mig}$ ) | 0.5                 |
| SLA Violation Threshold ( $\tau_{sla}$ )        | 0.3                 |
| SLA Success Bonus ( $\beta_{success}$ )         | 0.02                |
| SLA Violation Penalty ( $\beta_{violation}$ )   | 0.10                |
| Payment Dishonesty Penalty ( $\lambda$ )        | 0.25                |
| Payment SLA Penalty ( $\mu$ )                   | 0.10                |
| Passive Recovery Rate ( $r_{passive}$ )         | 0.01                |
| Passive Recovery Interval                       | 21 s                |
| <b>Task Model</b>                               |                     |
| Tasks per Edge Device                           | 20                  |
| Mean Inter-Arrival Time                         | 160 s               |
| Runtime Range                                   | 240–400 s           |
| Demand MIPS Range                               | 200–600 MIPS        |
| RAM Range                                       | 128–512 MB          |
| Bandwidth Range                                 | 150–550 Mbps        |
| Container Size Range                            | 200–420 MB          |

### 3.9 Metrics for Comparison

To understand how the proposed collaborative FT approach behaves compared to the baseline configuration, we rely on a set of metrics that capture how well the system maintains timeliness, how smoothly it handles migrations, and how stable it remains when faults occur. These metrics highlight the practical influence of gossip visibility, trust-aware bidding, and prediction-driven migration, and they help show how these components shape the flow of tasks through the fog layer.

**SLA violation rate.** Each task in the system is associated with a completion deadline, and a violation is recorded whenever the task finishes after this deadline. Since both predicted faults and unexpected degradations can delay execution or increase migration overhead, the SLA violation rate provides a direct indication of whether the proposed FT framework improves reliability under fault conditions. A lower violation rate typically reflects more stable execution.

**Average migration time.** This measures how long a container remains in the migration process, beginning at the pause event and ending when execution resumes at the destination. A container may experience several migrations during its lifetime, especially in environments with repeated or overlapping faults, and for this reason average migration time becomes an important metric to observe. It shows whether the FT mechanisms

keep recovery overhead manageable or whether repeated migrations accumulate into noticeable performance penalties. Lower values usually indicate smoother and more predictable recovery.

**Average makespan time.** Makespan represents the total duration a task spends in the system, including execution time, any delays caused by predicted or reactive faults, and all migration-related overhead. We compute this value for each task and then average across the workload. Makespan offers a broad view of end-to-end performance and helps reveal whether tasks are progressing efficiently despite migrations and disruptions.

**Total number of migrations.** To better understand how the system responds to faults, we track the total number of migrations and classify them into intra-fog, inter-fog, and cloud migrations. Intra-fog migrations reflect the system’s ability to recover locally. Inter-fog migrations show how often cooperation among fog nodes is required. Cloud migrations indicate situations where neither local recovery nor inter-fog support is feasible. Observing the distribution of these migration types helps compare the behavior of the proposed FT approach with the baseline configuration, particularly in terms of how often unnecessary or avoidable migrations occur.

Together, these metrics provide a comprehensive view of responsiveness, efficiency, and stability, allowing us to compare the proposed FT approach with the baseline configuration in a controlled and meaningful way.

**Algorithm 6** Collaborative Migration Workflow at Fog Node  $i$ 


---

```

1: Input: Container  $c$ , fault event  $f$ , servers  $S_i$ , state table  $StateTable_i$ 
2: Output: Container migrated to new location
3:
4: // Handle fault and checkpoint if possible
5:  $sourceServer \leftarrow$  server hosting container  $c$ 
6: if  $f$  is irrecoverable then
7:   Remove  $c$  from  $sourceServer$  without checkpointing
8:    $c.resetProgress()$ 
9: else
10:  Pause container  $c$ 
11:  Checkpoint  $c$  with current progress at time  $t$ 
12:  Remove  $c$  from  $sourceServer$ 
13: end if
14:
15: // Phase 1: Attempt intra-fog migration
16:  $localTarget \leftarrow$  default scheduler places  $c$  on a healthy server in  $S_i$ 
17: if  $localTarget \neq \text{null}$  then
18:   Resume execution of  $c$ 
19:   Record intra-fog migration
20:   return
21: end if
22:
23: // Phase 2: Inter-fog migration via reverse auction
24:  $Candidates \leftarrow$  Score all fog nodes using Algorithm 5
25: Limit  $Candidates$  to top  $N_{max}$  nodes
26:
27: // Request bids from candidates
28:  $Bids \leftarrow \emptyset$ 
29: for each fog node  $j \in Candidates$  do
30:   Send migration request to  $j$ 
31:   Await bid response from  $j$  (computed via Algorithm 2)
32:    $Bids \leftarrow Bids \cup \{\text{received bid}\}$ 
33: end for
34:
35: // Select winner using trust-adjusted bidding
36:  $winner \leftarrow \text{SelectWinner}(c, Bids)$  using Algorithm 3
37: if  $winner \neq \text{null}$  and  $winner \neq \text{cloud}$  then
38:   Initiate transfer of  $c$  to fog node  $winner$ 
39:   Measure actual bandwidth  $Actual_{bw}$  and migration time  $T_{mig}^{actual}$ 
40:   Winner's default scheduler places  $c$  on appropriate server
41:   Record inter-fog migration
42:   return
43: end if
44:
45: // Phase 3: Cloud fallback
46: if cloud is available then
47:   Initiate transfer of  $c$  to cloud
48:   Cloud's default scheduler places and resumes/restarts  $c$ 
49:   Record cloud migration
50: else
51:   Enqueue  $c$  for retry when resources become available
52: end if

```

---

▷ Work progress lost, restart required  
 ▷ Fault is recoverable

**Algorithm 7** Payment Settlement and Combined Trust Update

---

```

1: Input: Container  $c$ , winning fog node  $i$ , completion time  $T_{complete}$ , bid context
2: Output: Payment transfer and updated trust score
3:
4: // Apply passive trust recovery for all fog nodes
5: PassiveRecovery for all fog nodes at current time
6:
7: // Retrieve bid and actual metrics
8:  $Bid_{value} \leftarrow$  original bid value from fog node  $i$ 
9:  $Claimed_{bw} \leftarrow$  bandwidth claimed in bid
10:  $T_{mig}^{claimed} \leftarrow$  migration time claimed in bid
11:  $Actual_{bw} \leftarrow$  measured actual bandwidth during transfer
12:  $T_{mig}^{actual} \leftarrow$  measured actual migration time
13:
14: // Compute error metrics
15:  $e_{bw} \leftarrow \frac{|Claimed_{bw} - Actual_{bw}|}{\max(\varepsilon, Claimed_{bw})}$ 
16:  $e_{mig} \leftarrow \frac{|T_{mig}^{claimed} - T_{mig}^{actual}|}{\max(\varepsilon, T_{mig}^{claimed})}$ 
17:
18: // Compute SLA margin
19:  $m_{sla} \leftarrow \frac{D_{sla} - T_{complete}}{\max(\varepsilon, D_{sla})}$ 
20:
21: // Update trust using combined evaluation (Algorithm 4)
22:  $t_{updated}(i) \leftarrow \text{TrustUpdate}(i, c, Actual_{bw}, T_{mig}^{actual}, Claimed_{bw}, T_{mig}^{claimed}, T_{complete})$ 
23:
24: // Calculate payment with penalties
25:  $P_{lie} \leftarrow \lambda(e_{bw} + e_{mig})$ 
26:  $P_{sla} \leftarrow \mu \cdot \max(0, -m_{sla})$ 
27:  $Payment \leftarrow \max(0, Bid_{value} - P_{lie} - P_{sla})$ 
28:
29: // Execute payment transfer
30: if fog node  $i \neq$  owner of  $c$  and  $i \neq$  cloud then
31:   Debit  $Payment$  from owner's account
32:   Credit  $Payment$  to fog node  $i$ 's account
33: end if
34:
35: Record settlement metrics:  $(c.id, i, Bid_{value}, Payment, t_{updated}(i), m_{sla})$ 

```

---